

ORMs & Abstraction Layers

Simplifying Data Access

Object-Relational Mappers (ORMs) bridge the gap between object-oriented code and relational databases. Instead of writing SQL and manually mapping results to objects, you work with objects directly and let the ORM handle the translation.

What ORMs Do

```
// Without ORM
$result = $db->query("SELECT * FROM users WHERE id = 1");
$user = new User($result['id'], $result['name'], $result['email']);
```

```
// With ORM
$user = User::find(1);
```

ORMs handle:

- Query generation
- Result mapping
- Relationship traversal (`$user->posts`)
- Change tracking and persistence
- Schema migrations

Popular ORMs

Language	ORM	Notes
PHP	Eloquent, Doctrine	Eloquent emphasizes simplicity; Doctrine emphasizes purity
Python	SQLAlchemy, Django ORM	SQLAlchemy is more flexible; Django's is tightly integrated
JavaScript	Prisma, TypeORM, Sequelize	Prisma has excellent TypeScript support
Java	Hibernate, JPA	Enterprise standard
C#	Entity Framework	Microsoft's flagship ORM

The Trade-Off

ORMs abstract away SQL—sometimes too much. When performance matters or queries get complex, you may need to drop down to raw SQL. Good ORMs make this easy.

The danger is not understanding what's happening underneath. An innocent-looking loop can trigger hundreds of queries (the “N+1 problem”). Learn to read the SQL your ORM generates.